BULKAPI · IO_URING HTTP SERVER

Optimisations, one by one

What each change does, what it measured, and which ones are still open.

4 implemented **5** still open Measured on one 4-core laptop

STARTING POINT

Where bulkapi’s CPU time goes

Single requests are bound by the kernel. Our parser, router and handler together cost under 1 µs.

Bulk requests spend most of their time in our own code. That is where optimising pays.

Workload	Our code	Kernel	Kernel share
GET, no pipelining	0.62 µs	6.31 µs	91%
GET, pipeline of 8	0.38 µs	1.02 µs	73%
Bulk insert, 100 items	13.9 µs	12.9 µs	48%
Bulk get, 100 items	39.4 µs	15.9 µs	29%

CPU per request, io_uring backend, one worker, 64 connections

METHOD

How each change was measured

Same answers

A 34-line probe is run before and after. Responses must be byte-identical, including split packets, pipelining and a 20 MB upload.

CPU per request

One server thread, so server CPU time divided by requests is meaningful. Old and new binaries run in alternating rounds.

Honest about noise

The same binary varies by up to 2x between runs here. A difference under about 1.5x is not treated as real.

BACKGROUND

What io_uring is

io_uring is Linux’s interface for asynchronous input and output. The program and the kernel share two queues in memory, so they can exchange many requests without a system call for each one.

Submission queue

The program writes what it wants done: accept a connection, receive, send.

Completion queue

The kernel writes the result of each operation here when it finishes.

One call for many

A single system call hands over every queued request and collects every finished one.

With the older epoll interface every read and every send is its own system call: about twice the kernel time per request here

The event loop

```
for (;;) {
  io_uring_submit_and_wait(&ring, 1);
  io_uring_for_each_cqe(&ring, head, cqe) {
    seen++;
    u64 op = cqe->user_data & OP_MASK;
    Conn* c = (Conn*)(cqe->user_data & ~OP_MASK);
    if (op == OP_RECV)
      on_recv(c, cqe->res, cqe->flags);
    else if (op == OP_SEND)
      on_send(c, cqe->res);
  }
  io_uring_cq_advance(&ring, seen);
}
```

One call does both jobs. submit_and_wait hands the kernel everything queued since the last turn, then sleeps until at least one operation has finished.

Each completion says whose it is. user_data is a tag the program attached when it submitted: here the connection’s address, with the operation type in its lowest three bits.

cq_advance tells the kernel those results have been read.

Abridged from backend_uring_ms.cpp (accept handling left out)

The same loop in Java

```
for (;;) {
  enter(true);       // the one native call
  int head = cqHead.getInt(0);
  int tail = (int) INT.getAcquire(cqTail, 0);
  while (head != tail) {
    int off = (head & cqMask) * cqeSize;
    long ud = cqes.getLong(off + oUd);
    int res = cqes.getInt(off + oRes);
    head++;
    Conn c = conns.get((int) (ud >>> OP_BITS));
    // ... a receive or a send finished
  }
  INT.setRelease(cqHead, 0, head);
}
```

cqes is the kernel's completion queue, mapped into Java as a direct ByteBuffer. Java reads results straight out of shared memory.

enter() is the only call that leaves Java: one per turn, however many requests it covers.

The tag is an index, not a pointer: Java has no addresses, so user_data holds the connection's position in a list.

Abridged from UringBackend.java · liburing sets the ring up through a small JNI shim, native/uring_jni.c

BACKGROUND

Single-shot and multishot

Single-shot

One submission gives one completion. To receive the next request, the program must submit a new receive.

The program says which buffer to fill, so each connection needs its own.

`--backend uring`

Multishot

One submission stays armed and gives a completion every time data arrives. Nothing is resubmitted.

The kernel picks a buffer from a pool shared by all connections on the thread.

`--backend uring-ms`

Accepting connections was multishot in both backends from the start; the difference is in receiving

OPTIMISATION 1 · IMPLEMENTED

Multishot receive

What changed. One receive is armed per connection and keeps delivering data into a shared ring of buffers. Before, a receive was submitted again after every request.

Result. Cost per request stays flat as connections grow. It is cheaper than single-shot at 2,000 connections and dearer at 256.

Connections	Single-shot	Multishot
256	8.8–9.8 µs	15.5–19.5 µs
1,000	22 µs	15–18 µs
2,000	27–28 µs	14.5–14.7 µs

Server CPU per request, GET, no pipelining, one thread · --backend uring-ms

Arming a multishot receive

```
void prep_recv_multishot(Conn* c) {
  io_uring_sqe* e = sqe();
  io_uring_prep_recv_multishot(
      e, c->fd, nullptr, 0, 0);
  e->flags |= IOSQE_BUFFER_SELECT;
  e->buf_group = kBufGroup;
  io_uring_sqe_set_data64(e, (u64)c | OP_RECV);
  c->recv_armed = true;
}

// later, on each completion
if (!(flags & IORING_CQE_F_MORE))
  c->recv_armed = false;
```

No buffer is named. The buffer argument is null and the length is 0. The kernel will choose a buffer each time data arrives.

BUFFER_SELECT and buf_group say which shared pool to take that buffer from.

F_MORE on a completion means the receive is still armed. When it is missing, for example because the pool ran dry, the loop arms it again.

From backend_uring_ms.cpp · io_uring_prep_recv_multishot needs liburing 2.3 or newer and Linux 6.0 or newer

OPTIMISATION 2 · IMPLEMENTED

No buffer per connection

What changed. A request is parsed in place in the shared receive buffer, with no copy. A private buffer exists only while a request is incomplete, and is freed after.

Result. Memory per idle connection fell 14-fold. Speed is unchanged.

Before, per idle connection

4.54 KB

After, per idle connection

0.30 KB

Process memory growth, 20,000 connections idle after one request each · --backend uring-ms

How the memory was reduced

1. The cost before

Every connection owned a 16 KB buffer. Memory comes in 4 KB pages, and the first page is in use as soon as the buffer exists: 4 KB per connection.

2. Share the buffers

Data now lands in a pool of 1,024 buffers per thread. The pool costs about 4 MB once, however many connections there are.

3. Read it in place

The request is handled straight from the shared buffer, which is then handed back. The connection keeps only a record of about 95 bytes.

A private buffer is still created when a request arrives in pieces, and is freed once the request is complete.

Opening 20,000 connections added 80 MB before and 1.9 MB after · the kernel’s own 4 KB per socket is unchanged

Parsing in the shared buffer

```
void feed(Conn* c, char* p, size_t n) {
  if (c->in_len || c->sending) {
    stash(c, p, n);        // keep a copy
    if (!c->sending) run(c);
    return;
  }
  c->in = p;               // parse in place
  c->in_cap = c->in_len = n;
  process(*c, w);
  size_t left = c->in_len;
  c->in = nullptr;
  c->in_cap = c->in_len = 0;
  if (left) stash(c, p, left);
  after_process(c);
}
```

p is the kernel's shared buffer. In the common case the connection simply points at it, the request is handled, and the pointer is cleared. Nothing is copied.

left is what remains of an incomplete request. Only that is copied to a private buffer.

The first branch covers a connection that already holds input or is mid-reply: its bytes must be kept, because p goes back to the kernel on return.

Abridged from backend_uring_ms.cpp (output-buffer pooling and backpressure left out)

OPTIMISATION 3 · IMPLEMENTED, NO MEASURABLE GAIN

Write each response once

What changed. Handlers append the body straight into the output buffer after a reserved gap, and the header is filled in afterwards. Before, the body was built separately and then copied.

Result. The copy is gone, but it was not where the time went.

Bulk get	Before	After
100 items, run 1	44.6 µs	40.9 µs
100 items, run 2	46.0 µs	44.8 µs
1,000 items	344–363 µs	347 µs

Body first, header afterwards

```
size_t start = begin_response(c.out);
Res res{200, "application/json", c.out};
kRouteHandler[ridx](req, res);
finish_response(c.out, start, res.status,
                res.ctype, !keep_alive);

// inside finish_response
if (body > kSmallBody) {
  memset(h + n, ' ', pad);  // pad the header
  n += pad;
}
if (n < kHdr) {             // close the gap
  memmove(h + n, h + kHdr, body);
  o.resize(start + n + body);
}
```

begin_response leaves a 160-byte gap in the output buffer. The handler’s body lands right after it, because res.body *is* the output buffer.

Large body (over 4 KB): it stays put. The header is padded with spaces so it fills the gap exactly.

Small body: it is moved down against the header. Copying a few bytes is cheaper than sending 90 spaces.

Abridged from http.cpp · the padding goes in front of the Content-Length value, which HTTP permits

Body first, in Java

```
int start = out.begin();
route.handler.handle(w, out);
out.finish(start, w.status, w.ctype, !keepAlive);

// inside Out.finish
boolean small = bodyLen <= SMALL;
if (!small)                 // pad the header
  for (; pad > 0; pad--) h[n++] = ' ';
b.position(start);
b.put(h, 0, n);
if (small) {                // close the gap
  ByteBuffer src = b.duplicate();
  src.limit(end).position(bodyStart);
  b.put(src);
}
```

Same design as the C++ change. The handler writes into the output buffer after a 160-byte gap, and the header is filled in afterwards.

The buffer is off-heap. b is a direct ByteBuffer, so the kernel sends from it without an extra copy out of the Java heap.

b.put(src) on a view of the same buffer is Java's way to move the small body down.

Abridged from Http.java and Out.java · the Java port was written this way from the start

OPTIMISATION 4 · IMPLEMENTED, OFF BY DEFAULT

Batching under high load

What changed. When the previous loop turn handled 32 or more completions, the loop can wait a short time for more instead of waking for the first one.

Result. No consistent gain at any load tried. A busy loop already picks up many completions per wake-up.

Connections	Off	200 µs	1,000 µs
16	19–20	17.5	18
256	14–15	13–14	14–17
2,000	14–16	14–20	15–16

Server CPU per request in µs, GET, one thread · BULKAPI_BATCH_US, uring-ms backend

Waiting longer only when busy

```
if (batch_us > 0 && last >= kBusy) {
  __kernel_timespec ts{0, batch_us * 1000};
  rc = io_uring_submit_and_wait_timeout(
      &L.ring, &cqe,
      std::min(last / 2, 64u),
      &ts, nullptr);
} else {
  rc = io_uring_submit_and_wait(&L.ring, 1);
}

// after the completions are handled
last = seen;
```

last is how many completions the previous turn handled. kBusy is 32: fewer than that and the loop wakes for the first completion, as before.

When busy, it asks for half as many again, at most 64, and gives up waiting after batch_us microseconds.

The reference always waits for 64 completions or 2 ms once a turn has seen 8, which is what costs it at low connection counts.

From backend_uring_ms.cpp · batch_us comes from the BULKAPI_BATCH_US environment variable and defaults to 0, which is off

EVIDENCE FOR WHAT COMES NEXT

Against the reference without its database

bulkapi wins at low connection counts and on pipelining, by 6x to 11x.

The reference still wins at 2,000 connections and on bulk get. Those two gaps are what the open optimisations target.

Requests per second	Reference	uring	uring-ms
GET, 16 connections	6.5k	74k	51–72k
GET, 256 connections	84k	82k	63k
GET, 2,000 connections	62k	30k	46k
GET, pipeline of 8	66k	390–640k	410–440k
Bulk get, 100 items	36k	22k	13–19k

All three single-threaded, same session, two rounds · reference-nodb is the reference minus SQLite and JWT

Three Java backends compared

Requests per second	C++ uring	Java uring	Java epoll	Java sockets
GET, no pipelining	251–421k	267–271k	217–223k	73–132k
GET, pipeline of 8	1.04–1.35M	1.20–1.22M	0.71–0.94M	0.53–0.68M
Bulk insert, 100 items	46–60k	55–65k	46–50k	55–56k
Bulk get, 100 items	43–54k	62–63k	46–47k	55–58k
Memory after the run	20 MB	349–364 MB	335–346 MB	396–400 MB

Among the Java backends, io_uring is fastest on every workload, epoll is about 20% behind, and blocking sockets fall furthest behind on small requests.

Against C++, Java io_uring is level within this machine's noise, at about 18 times the memory.

Two rounds, range shown · 4 threads, 256 connections (64 for bulk) · JDK 11, 1 GB starting heap · 0 errors

Java epoll backend

```
for (;;) sel.select(this::onKey);

private void onKey(SelectionKey key) {
  Conn c = (Conn) key.attachment();
  if (c == null) { accept(); return; }
  c.ensureRoom();
  ByteBuffer in = c.in;
  in.limit(in.capacity()).position(c.inLen);
  int n = c.ch.read(in);
  if (n < 0) { drop(c); return; }
  c.inLen += n;
  Http.process(c, w);
  if (flush(c)) return;
  c.writing = true;
  key.interestOps(SelectionKey.OP_WRITE);
}
```

A Selector is epoll. On Linux, Java's selector is built on it. It reports which sockets are ready; the program then reads and writes them itself.

select(this::onKey) calls the method for each ready socket without building a set of keys first.

OP_WRITE is requested only when a reply does not fit in the socket in one go.

Abridged from EpollBackend.java (the write-ready branch and error handling left out) · one selector per thread

Java blocking-sockets backend

```
while (!c.wantClose) {
  c.ensureRoom();
  ByteBuffer in = c.in;
  in.limit(in.capacity()).position(c.inLen);
  int n = ch.read(in);    // blocks this thread
  if (n < 0) break;
  c.inLen += n;
  Http.process(c, w);
  ByteBuffer b = c.out.b;
  if (b.position() == 0) continue;
  b.flip();
  while (b.hasRemaining()) ch.write(b);
  c.out.clear();
}
```

One thread per connection. The thread sleeps inside read() until the client sends something. There is no event loop at all.

The same parser. Http.process is shared by all three Java backends; only the way bytes arrive and leave differs.

The cost: 256 connections means 256 threads competing for 4 cores, which is why it falls behind on small requests.

Abridged from SocketsBackend.java · threads come from a cached pool, so they are reused across connections

The store the C++ server should copy

```
private long[] keys = new long[16];
private byte[][] vals = new byte[16][];

byte[] get(long k) {
  int m = keys.length - 1;
  for (int i = slot(k, m); vals[i] != null;
       i = (i + 1) & m)
    if (keys[i] == k) return vals[i];
  return null;
}

private int slot(long k, int mask) {
  return (int) ((k * MIX) >>> 20) & mask;
}
```

Two flat arrays. Keys are plain 64-bit numbers side by side, with no object and no pointer per entry. A null value marks an empty slot.

A lookup walks forward from the key's home slot until it finds the key or an empty slot. Neighbouring slots share a cache line.

This is optimisation 5, already built here. The C++ server still uses a node-based map.

From Store.java · 256 such shards, each locked separately · Java bulk get measured 62–63k req/s against C++ 43–54k

OPTIMISATION 5 · NOT DONE

Replace the store’s hash map

Today

Each shard is a std::unordered_map. Every new item allocates a node, and every lookup follows a pointer to it.

Proposed

An open-addressing table: keys and values in flat arrays, no node per entry, short values stored inline.

Why expect a gain

The reference and the Java port both use this layout. The reference’s bulk get costs about 30 µs against our 43.

OPTIMISATION 6 · NOT DONE

Lock once per shard, not per item

Today

A bulk request of 100 items takes and releases a spinlock 100 times, once for each item.

Proposed

Group the batch by shard first, then take each shard’s lock once and handle all of its items together.

Trade-off

Bulk get must still return items in request order, so grouping needs a second pass. It pays most with several threads.

OPTIMISATION 7 · NOT DONE

Faster JSON output

Today

Escaping checks one byte at a time, and each item is built from several small appends that may each regrow the buffer.

Proposed

Scan 16 bytes at a time for a character that needs escaping, copy clean runs in one go, and reserve the response size up front.

Why expect a gain

Bulk get is 70% our own code, and its output is almost all escaped values. The reference already scans 16 bytes at a time.

Expected, not measured · json.cpp, handlers.cpp

OPTIMISATION 8 · NOT DONE

One system call per request

Today. With a single client, the single-shot backend enters the kernel twice per request: once for the receive completion, once for the send completion.

Proposed. Link the send to the next receive and suppress the send’s completion, so one entry covers both.

Kernel entries per request, 1 client

2.0

Kernel entries per request, 64 clients

0.03

Counted with strace, which inflates batching · the proposal itself is not measured · backend_uring.cpp

OPTIMISATION 9 · NOT DONE

Less kernel work per operation

Change	What it saves	Where it helps
Direct descriptors	A file-table lookup on every receive and send	Every request
Registered ring	A file lookup on every kernel entry	Low concurrency
No-delay on the listener	One system call per accepted connection	Short-lived connections
Zero-copy send	The copy of the response into the kernel	Large responses, not loopback
Idle timeout	Memory held by dead or stalled clients	Many connections

The kernel is 73–91% of the cost of a small request, so this is the only route to a faster single GET. Each item is small, and none has been measured here.

BEFORE MORE SPEED · NOT DONE

Three defects to fix first

Unicode escapes corrupt data

Inserting “café” stores “cafu00e9”. The backslash is dropped. The Java port and the reference decode it correctly.

Allocations are unchecked

A failed allocation for a 64 MB body is a crash. The reference checks every one.

Header values are not trimmed

A Content-Length with a trailing space is answered 400. The Java port and the reference accept it.

All three confirmed against the running C++ server · json.cpp, conn.cpp, http.cpp

SUMMARY

Every optimisation and its status

#	Optimisation	Status	Measured effect
1	Multishot receive	Done	Flat 15 µs per request; better at 2,000 connections, worse at 256
2	No buffer per connection	Done	Idle memory 4.54 KB to 0.30 KB per connection
3	Write each response once	Done	None beyond noise
4	Batching under high load	Done, off	None consistent
5	Replace the store’s hash map	Open	Not measured
6	Lock once per shard	Open	Not measured
7	Faster JSON output	Open	Not measured
8	One system call per request	Open	Not measured
9	Less kernel work per operation	Open	Not measured

One of four implemented changes gave a clear win; one is a trade-off; two changed nothing measurable

RECOMMENDED ORDER

What to do next

1. Fix the Unicode escape defect: it corrupts stored data.
2. Replace the store’s hash map and speed up JSON output together, then measure bulk get.
3. Rerun everything with the load generator on a second machine.
4. Only then consider the kernel-side items.

Two of four changes so far measured nothing. Each next step should be benchmarked before and after, and dropped if it does not show.

Details: COMPARISON.md, bulkapi/RESULTS.md, reference-nodb/README.md